

From HTML/CSS to Component-Based React

Components • Pages • JSX • Import & Export • Real-World Migration



Today's Goal

Students will understand React application structure, components, pages, JSX, and import/export, then apply those ideas by migrating the existing Health Center website from HTML/CSS into React.

Teaching principle: explain the concept → show a tiny example → immediately apply it to the Health Center project.

Class Flow

01	Recap	Why React and what problem it solves
02	Structure	index.html → main.jsx → App.jsx
03	Components	UI building blocks and component thinking
04	Pages	Complete screens vs UI sections
05	JSX	HTML-like syntax inside JavaScript
06	Modules	Why import/export are needed
07	Migration	Convert Health Center into React components

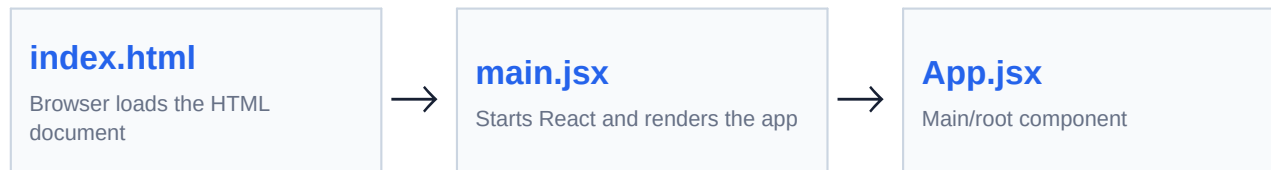
01. React Recap & Application Structure

Why React?

Traditional websites can become difficult to maintain as the UI grows. Navigation, cards, buttons and sections may be repeated, and updating shared UI patterns can require changes in multiple places. React encourages a component-based approach so the interface can be divided into manageable pieces.

Key idea: React is a JavaScript library for building user interfaces. Its component model helps developers organize UI as a collection of independent building blocks.

The React entry flow



index.html

- The browser loads an HTML document first.
- It contains the root element where React will mount the application.

main.jsx

- The React entry point that connects the React application to the HTML root element.
- It renders the main application component.

App.jsx

- The main/root React component where the application UI starts.
- It composes other components together.

Important browser explanation

The browser understands HTML, CSS and JavaScript. JSX is transformed by the React toolchain into JavaScript that can create the UI. **main.jsx** starts the React application and renders it into the HTML element in **index.html**.

Mental model: HTML provides the place → main.jsx starts React → App.jsx describes the application UI.

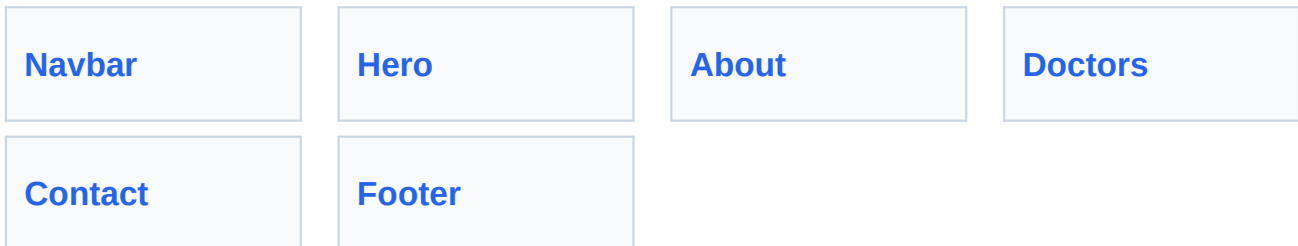
02. Components, Pages, JSX & Modules

What is a Component?

A component is a small, independent building block of a user interface. In modern React, components are commonly written as functions that return JSX.

A component can be reusable, but it does not have to be reused. A meaningful UI section can be a component simply because separating it makes the application easier to understand and maintain.

Health Center example



Pages vs Components

PAGE	COMPONENT
Represents a complete screen or route.	Represents a UI building block or section.
Examples: HomePage, AdminPage, SettingsPage.	Examples: Navbar, DoctorCard, Sidebar, Footer.
Often assembled from many components.	May be reused in one page or many pages.

JSX

JSX lets us write HTML-like UI inside JavaScript. It makes component structure easier to read while still allowing JavaScript expressions where needed.

```
function Hero() {  
  return <h1>Welcome to Health Center</h1>;  
}
```

Import & Export

As the application is split into files, one file needs a way to expose a component and another file needs a way to use it. export makes a value available from a module; import brings it into another module.

```
Navbar.jsx → export → App.jsx → import → <Navbar />
```

03. Migration Workshop — Health Center

Starting point

The existing project is a single HTML/CSS website with a Navbar, Hero, About, Doctors, Contact and Footer. The goal is not to redesign it. The goal is to preserve the UI while changing the architecture to React.

Existing HTML section	React component
Navigation	Navbar.jsx
Hero section	Hero.jsx
About section	About.jsx
Doctors section	Doctors.jsx
Contact section	Contact.jsx
Footer	Footer.jsx

Target React structure

```
src/  
├── components/  
│   ├── Navbar.jsx  
│   ├── Hero.jsx  
│   ├── About.jsx  
│   ├── Doctors.jsx  
│   ├── Contact.jsx  
│   └── Footer.jsx  
├── assets/  
│   ├── App.jsx  
│   ├── main.jsx  
│   └── index.css
```

Migration sequence

1. **Create the components** — Move each meaningful website section into its own JSX component.
2. **Move the styling** — Bring the existing CSS into the React project without changing the design unnecessarily.
3. **Fix JSX syntax** — Convert HTML syntax where React requires JSX conventions, such as class → className.
4. **Export components** — Expose each component so App.jsx can use it.
5. **Import components** — Import the components into App.jsx.
6. **Compose App.jsx** — Place Navbar, Hero, About, Doctors, Contact and Footer in the required order.
7. **Run & compare** — Check the React version against the original website and verify that the UI still works.

End-of-class mental model



Do not introduce React Router, useState, useEffect, props in depth, API calls or state-management libraries today. Those topics will make more sense after students are comfortable creating and composing components.

Success check: every student should be able to explain why the website was split into components, what App.jsx does, why JSX is used, and why import/export are needed.